

How Zcash Hides a Transaction

and what the zero-knowledge proofs actually prove

m@rek.onl

What a transparent ledger publishes

- A transparent ledger (Bitcoin, Zcash's transparent pool) records, for every unspent output, *its owner and its denomination* — and a payment publicly transfers an output from one owner to another.
- Per output, forever: owner, amount, counterparty.
- That is enough raw material for address clustering, amount tracing, and the full transaction graph.

A shielded payment must achieve the same effects — transfer of value, prevention of double-spending, conservation of supply — while disclosing none of them.

Four requirements, four mechanisms

(the talk's roadmap)

1. Represent value hidden yet provable → notes + note commitments
2. Prove possession of a real, unspent note without identifying which commitment tree → the
3. Prevent double-spending without a public spent-list → nullifiers
4. Prove no value created without revealing amounts → value commitments + binding signature

One zero-knowledge proof per bundle makes all four hold *at once* — the Orchard *Action statement*, the centre of this talk.

The data model

A note

$$\mathbf{n} := (\text{addr}, v, \rho, \psi, \text{rcm})$$

- Address $\text{addr} = (d, \text{pk}_d)$ — an 88-bit diversifier and a transmission key; value $v \in \{0, \dots, 2^{64} - 1\}$ zatoshi.
- The randomness $\rho, \psi \in \mathbb{F}_{p_{\text{Pallas}}}$ exists solely to render the eventual *nullifier* unique and unpredictable; $\text{rcm} \in \mathbb{F}_{p_{\text{Vesta}}}$ is the commitment trapdoor.
- A note carries randomness whose purpose is downstream.

The note itself never touches the chain.

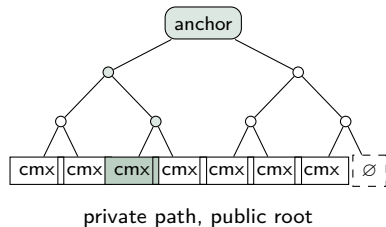
The chain stores a sealed envelope

$$\text{cm} := \text{NoteCommit}_{\text{rcm}}(g_d^* \parallel pk_d^* \parallel \text{LE}_{64}(v) \parallel \text{LE}_{255}(\rho) \parallel \text{LE}_{255}(\psi)) \in \mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$$

- On chain goes only $\text{cm}_x := \text{Extract}(\text{cm})$, the x -coordinate — one field element.
- *Perfectly hiding*: a uniform blinder masks the hash completely — privacy survives even an unbounded future adversary.
- *Computationally binding*: reopening one commitment as two notes solves discrete log on Pallas. (That direction, never the reverse — and no scheme is perfect at both.)
- The hash is Sinsemilla, chosen for *in-circuit* cost: one table lookup per 10-bit chunk — because the spender must recompute cm inside the proof.

One tree hides them all

- Every cmx ever created, in creation order, in one append-only Merkle tree of fixed depth 32.
- The root after block h is the *anchor* of block h .
- A spend proves membership under some anchor; the path is private — *the anonymity set is the entire tree*.
- Appending alters no old root, so **old anchors never expire**; the nullifier, not the anchor, prevents double-spends.
- Wallets still anchor near the tip — by policy: the newest tree is the largest anonymity set, and an odd anchor fingerprints the spender.



The nullifier: a one-time serial number

$$\text{nf} := \text{Extract}([F_{\text{nk}}(\rho) + \psi] G^{\text{nf}} + \text{cm}) \in \mathbb{F}_{\rho_{\text{Pallas}}}$$

with $F_{\text{nk}}(\rho) = \text{PoseidonHash}(\text{nk}, \rho)$ and G^{nf} a nothing-up-my-sleeve generator.

- Two requirements that pull in opposite directions:
- *Deterministic to the network* — every ingredient is fixed once the note exists, so spending twice yields the same nf twice, and the network rejects the second.
- *Pseudorandom to everyone else* — the masked term keeps the tag unlinkable to the commitment it retires (DDH on Pallas + the PRF).
- Chained uniqueness: each new note is created with $\rho_{\text{new}} := \text{nf}_{\text{old}}$, so nullifier uniqueness on chain forces ρ -uniqueness globally (defeats Faerie Gold forgeries).
- The nullifier key nk is deliberately not the spend key: viewing can be delegated without spending.

The mental model: spend = prove membership + reveal serial

$$\mathcal{R}_{\text{mem}} = \left\{ \left(\underbrace{\text{rt}}_{\text{public}}; \underbrace{(\nu, i, \pi)}_{\text{private}} \right) : \text{Verify}(\text{rt}, i, \nu, \pi) = \text{accept} \right\}$$

1. Locate your leaf, its path, its position in the public tree.
2. Choose a recent anchor that contains it.
3. Prove in zero knowledge: membership under the anchor, note well-formedness, correct nullifier derivation.
4. Publish the proof and the nullifier — *but neither the leaf, nor its position, nor the path.*

The root must be public — everyone verifies against agreed state.

The path must be private — nobody links the spend to a leaf.

Anatomy of an Action

Every Action spends *one* old note and creates *one* new note — a deliberately fixed shape that conceals whether it is “really” a spend, an output, or both.

Hidden (witness)

- the old note and the new note
- keys, Merkle path + position
- randomisers α , rcv

Public (per Action)

(anchor, cv^{net} , nf_{old} , rk , cmx)

plus the ciphertexts (epk , C_{enc} , C_{out})

Per *bundle*: one anchor shared by every Action, one aggregate Halo 2 proof over all Actions, one SpendAuthSig per Action, one declared net flow v_{balance} + one binding signature.

What the proof actually proves

The Action statement

The SNARK proves, with **public inputs**

(anchor, cv^{net} , nf_{old} , rk, cmx, enableSpend, enableOutputs, disableCrossAddress)

and the note, keys, Merkle path, randomiser α , and trapdoor rcv as **private witness**, the conjunction A1–A10.

- Per Action the verifier sees *ten field elements*; everything else is witness. (Curve points enter as coordinate pairs.)
- Checks A1–A9 are the NU5 statement; A10 arrives with NU6.3 (*Ironwood*).
- The next six slides are the statement in full — modulo one honest footnote: three checks are proved in $\perp$ -weakened form, noted where they occur.

A1/A2 — both notes really are notes

Old note opens the commitment the tree holds:

$$\text{cm}_{\text{old}} = \text{NoteCommit}_{\text{rcm}_{\text{old}}}(\text{g}_d^* \parallel \text{pk}_d^* \parallel \text{LE}_{64}(\nu_{\text{old}}) \parallel \text{LE}_{255}(\rho_{\text{old}}) \parallel \text{LE}_{255}(\psi_{\text{old}}))$$

New note, with $\rho_{\text{new}} := \text{nf}_{\text{old}}$:

$$\text{Extract}(\text{NoteCommit}_{\text{rcm}_{\text{new}}}(\dots)) \in \{\text{cmx}, \perp\}$$

(Both checks allow a $\perp$ escape for Sinsemilla's incomplete-addition exceptions — a disjunction costing only a negligible soundness term.)

- **Prevents:** opening one leaf as two different notes (binding = DL), and publishing a cmx that opens to nothing.
- The forced chaining $\rho_{\text{new}} := \text{nf}_{\text{old}}$ makes every note's ρ globally unique — the chain already enforces nullifier uniqueness.

A3 — the spent note is a real leaf

Recompute the root from $\text{Extract}(\text{cm}_{\text{old}})$ up the witnessed path of length 32; then

$$v_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$$

- **Prevents:** spending a note nobody ever created.
- The gate by v_{old} permits a *zero-value dummy* spend to skip membership — how an Action pads with no real input.
- **The escape hatch is arithmetic, not a flag:** value zero *is* the permission to skip, and a zero-value note mints nothing.
- Each tree layer is hashed with its height tag, so a path at one height cannot replay at another.

A5 — the published serial number is the right one

$$\text{nf}_{\text{old}} = \text{Extract}([F_{\text{nk}}(\rho_{\text{old}}) + \psi_{\text{old}}] G^{\text{nf}} + \text{cm}_{\text{old}})$$

- **Prevents:** emitting a fresh random “nullifier” per spend — which would allow unlimited double-spending.
- With A5, nf is a deterministic function of the note; the second spend collides and the network rejects it.
- Cross-note collisions reduce to discrete log on Pallas, with Poseidon treated as a PRF.

The double-spend *lookup* is not here: whether nf was already seen concerns global history — the one check that cannot sit inside a proof. The node does the set lookup; the proof guarantees the tag is honest.

A6/A7 — one identity signs, views, and nullifies

Spend authority: the published signature key is a fresh blinding of the owner's,

$$rk = ak + [\alpha] G^{\text{SpendAuth}}$$

Address integrity: the keys, the address, and the nullifier key belong to one identity,

$$ivk = \perp \text{ or: } ivk = \text{Commit}_{r_{ivk}}^{ivk}(\text{Extract}(ak), nk) \text{ and } pk_d = [ivk] g_d$$

(The $\perp$ -weakened form again, as in A1/A2.)

- **Prevents:** spending someone else's note, and deriving a nullifier from an unrelated nk (which would let one note yield many nullifiers).
- The same ak that A6 re-randomises is welded, through a binding commitment, to the nk of A5 and to the address of the spent note.
- Fresh α per Action makes rk *statistically* unlinkable across spends.

A4 — the value delta is committed, in range

$$cv^{\text{net}} = [v_{\text{old}} - v_{\text{new}}] V^{\text{Orchard}} + [rcv] R^{\text{Orchard}}, \quad v_{\text{old}}, v_{\text{new}} \in [0, 2^{64})$$

with V, R independent nothing-up-my-sleeve base points.

- **Prevents:** committing to anything but the true net change — and *overflow minting*: the sums to come live in a 255-bit field, so unconstrained values could wrap subtraction into free money. The 64-bit range checks close that.
- Each Action publishes one Pedersen envelope holding its value delta; nobody can read it, *everyone can add them* — the balance story, two slides ahead.

A8/A9/A10 — consensus gates

$$v_{\text{old}} \cdot (1 - \text{enableSpends}) = 0 \quad v_{\text{new}} \cdot (1 - \text{enableOutputs}) = 0$$

- **Prevents:** smuggling real value through a context that forbids it — a coinbase must set $\text{enableSpends} = 0$, so only zero-value dummies pass.
- From NU6.3, four more product gates of the same shape ($\delta := \text{disableCrossAddress}$) force the new note's receiver to equal the spent note's, as full curve points — sealing the legacy Orchard pool to self-transfers and turnstile exits.

Statement map: A1–A2 represent notes, A3 proves membership, A5 (with the chain's nullifier set) prevents double-spending, A4 conserves value, A6–A7 authorise the spender; A8–A10 are consensus-level gating.

What the statement does *not* include

- **Double-spend freshness** — the nullifier-set lookup concerns global history; the node does it.
- **Anchor validity** — the proof ties root to leaf; the node checks the root was ever a main-chain anchor.
- **Balance across Actions, fees, transparent flows** — pure public-key arithmetic, next slide.
- **Authority at spend time** — the SpendAuthSig verifies outside the circuit, under rk, over the transaction digest.
- **The ciphertexts** — entirely unproven; the recipient's own trial-decryption checks close forgery there.

*The circuit proves each Action is internally honest;
two signatures and two set lookups make the ledger honest.*

Balance without amounts

A public key only a balanced transaction can know

Envelopes add. Summing every Action's commitment in a bundle:

$$\sum_{i=1}^m cv^{\text{net},i} = \left[\sum_i (v_{\text{old},i} - v_{\text{new},i}) \right] V^{\text{Orchard}} + [\text{bsk}] R^{\text{Orchard}}, \quad \text{bsk} := \sum_i rcv_i$$

The network subtracts the *declared* net flow:

$$bvk := \sum_i cv^{\text{net},i} - [v_{\text{balance}}] V^{\text{Orchard}} \quad \text{balanced} \iff bvk = [\text{bsk}] R^{\text{Orchard}}$$

- If the books balance, the V -component cancels and bvk is a key whose secret the author knows; if not, knowing its discrete log would break independence of V and R .
- Balance is proved by *signing* with it: the 64-byte binding signature is a Schnorr proof of knowledge of bsk .
- Nobody chooses bvk — the transaction's own commitments force it, and the verifier recomputes it from public data.

The signature layer: one scheme, two meanings

Both signatures are RedPallas — verify $[s]P = R + [c]X$ with $c = H^*(R \parallel X \parallel \text{sighash})$ — differing only in the base point and in what knowing the secret *means*:

	base P	key X	knowing the secret means
SpendAuthSig (per Action)	$G^{\text{SpendAuth}}$	rk	authority over the spent note
binding sig (per bundle)	R^{Orchard}	bvk	the accounts balance

- Re-randomisation: $\text{rsk} := \text{ask} + \alpha$, $\text{rk} = [\text{rsk}] G^{\text{SpendAuth}}$ — for uniform α the published key is uniform *independently* of ak : the *key's* unlinkability is information-theoretic (the full spend also needs the proof and signature to leak nothing, and α to stay secret). The circuit (A6/A7) vouches the shift started from the note owner's key.
- Both sign the ZIP-244 sighash — a digest covering every Action, the flags, the balance, the anchor, and the network-upgrade ID: no field changes, no Action transplants, no cross-fork replay.

Three layers of trust

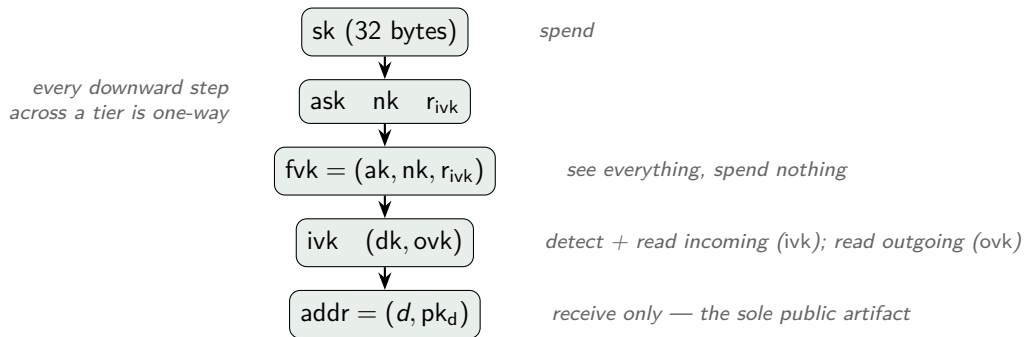
Halo 2 circuit	homomorphic sum	RedPallas signatures
per-Action facts A1–A10 cannot see across Actions	hidden deltas = declared flow cannot tell deltas are well- formed	authority + balance-knowledge cannot trace rk to an owner

The node's complete checklist — never seeing a note: proof · anchor is a known main-chain root · nf not in the nullifier set · one SpendAuthSig per Action + the binding signature. Then two state changes: the nullifier set gains nf_{old} , the tree gains cmx_{new} .

“The chain has gained one nullifier and one commitment; an observer learns that an Orchard payment occurred and nothing else — not sender, recipient, or amount.”

Keys: who can see what

The key ladder: one secret, graded capabilities



The full viewing key sees incoming, outgoing, *and spent-status*: holding nk it computes each note's nullifier and matches the chain — full balance history, no spend authority.

One key, 2^{88} unlinkable addresses

$$\begin{aligned}d_j &:= \text{FF1-AES256}_{\text{dk}}(j), & \text{pk}_d &:= [\text{ivk}] \text{g}_d, \\ \text{g}_d &:= \text{GroupHash}^{\text{Pallas}}(\text{z.cash:Orchard-gd}, d_j)\end{aligned}$$

- A keyed permutation of the index space, not a counter and not a hash:
- *unlinkable* — consecutive indices scatter over $\{0, 1\}^{88}$; consecutive diversifiers would reveal a shared owner;
- *bijective* — never collides, and the wallet inverts it to recover the index behind any of its addresses, storing no per-address state;
- *total* — every one of the 2^{88} diversifiers is valid (Sapling rejected about half; Orchard removed the loop).
- Give each payee their own address; all funds land in one wallet; linking two addresses of one owner is a DDH problem.

How the recipient finds the money

Nothing on chain marks whose output is whose — *detection is successful decryption*.

$$\text{epk} := [\text{esk}] g_d, \quad [\text{ivk}] \text{epk} = [\text{ivk}][\text{esk}] g_d = [\text{esk}] \text{pk}_d = s \rightarrow K_{\text{enc}}$$

- A fresh one-shot Diffie–Hellman per note; the wallet trial-decrypts every Orchard output; the AEAD tag makes wrong guesses fail cleanly.
- Before accepting, the wallet re-derives esk and checks $[\text{esk}]g_d = \text{epk}$, then recomputes the commitment against the on-chain cmx — forgery closes here, outside the SNARK.
- A second ciphertext C_{out} , sealed under a key derived from ovk, lets the *sender or an auditor* recover the same note.
- Light clients scan on 52-byte compact prefixes — lead byte, diversifier, value, note seed.

The privacy sliders

slider	artifact	grants
spend	sk	move the money
full view	fvk	incoming + outgoing + spent-status (auditor)
incoming view	ivk	detect and read incoming only (processor)
address	(d, pk_d)	receive; 2^{88} unlinkable ones per key

- “Viewing” names a *capability*, not publicity: every viewing key is secret material, disclosed deliberately, to confer exactly that visibility. Only the address is public.
- Zero disclosure is the default: a validating node holds no viewing key, cannot trial-decrypt, and never learns amounts, addresses, or memos.

The proving machine

What a zk-SNARK promises

- *Complete* — an honest prover who knows the witness always convinces the verifier.
- *Knowledge sound* — “knowledge” is meant literally: a satisfying witness could in principle be *extracted* from any prover who succeeds.
- *Zero-knowledge* — the verifier finishes convinced yet learns nothing beyond the bare truth of the claim.

“Soundness protects against false statements; knowledge soundness protects against provers who do not actually know what they claim.”

For Zcash: whoever produced this proof *possesses* a real note and its authority — that is why a node may safely act on it.

The trusted-setup problem, stated precisely

Pairing-era systems need a structured reference string built from a secret τ :

$$\text{pp} = ([\tau^0]G, [\tau^1]G, \dots, [\tau^d]G, \hat{G}, [\tau]\hat{G}), \quad \tau \text{ discarded}$$

- The trapdoor's power: anyone who learns τ can **forge proofs of false statements — counterfeit ZEC undetectably**.
- What it can *not* do: break privacy. The trapdoor's power is forgery, never decryption or linking; a leaked ceremony secret would have let someone print undetectable ZEC — and deanonymised no one.
- Mitigation: multi-party ceremonies — Sprout 2016, a six-party MPC; Sapling 2018, a large MPC whose universal first phase was the many-participant Powers of Tau (Groth16 still needs a per-circuit phase) — sound if *one* participant destroyed their share.
- Security of the money supply reduced to: “at least one of these people deleted a number.”

Three eras

	Sprout (2016)	Sapling (2018)	Orchard (2022–)
SNARK curve	BCTV14 BN254	Groth16 BLS12-381 + Jubjub	Halo 2 Pallas + Vesta
trusted setup	per-circuit (6-party)	per-circuit (large MPC)	none
in-circuit hash	SHA-256	Pedersen	Sinsemilla
proof size	~290 B	192 B	~5 kB (one-Action bundle)
verification	constant	constant (3 pairings)	$O(\log n)$ amortised
recursion	no	no	yes (Pasta cycle)

- A flaw in BCTV14 discovered in 2018 would have allowed undetected counterfeiting; the Sprout pool was deprecated and its value migrated — through a public turnstile.
- The realised risk came from a *proof-system bug*, not from toxic waste — even a perfectly honest ceremony would not have prevented it.

PLONKish: computation as a polynomial identity

Five loss-free translations: circuit $\rightarrow$ grid of gate-rows $\rightarrow$ column polynomials over roots of unity $\rightarrow$ “every row obeys its gate” $\rightarrow$

$$G(X) = t(X) Z_H(X), \quad Z_H(X) := X^n - 1$$

$\rightarrow$ *one random evaluation*, with binding commitments so the prover cannot adapt:

$$\Pr_z[\text{cheat accepted}] = \Pr_z[D(z) = 0] \leq \frac{\deg D}{|\mathbb{F}|} \approx 2^{-240}$$

- A correct computation *is* a polynomial identity; an incorrect one cannot be — and one random point tells them apart.
- The Orchard Action circuit: $2^{11} = 2048$ rows, ten advice columns, one lookup table for Sinsemilla’s 10-bit chunks.

Halo 2: why there is no ceremony

Commit with Pedersen over public-coin generators (hash-to-curve — no trapdoor *exists*):

$$\text{Commit}(p; r) := \langle \vec{a}, \vec{G} \rangle + [r] H$$

- Opening proofs by logarithmic halving: $O(\log d)$ group elements, assumption discrete log only.
- The price: one linear-time multi-scalar multiplication sits at the end of verification.
- The Halo trick (*accumulation*): that check is itself a commitment opening of a structured polynomial — so defer it into a running accumulator and decide once. A node batching many proofs pays the linear work once per batch.
- Causality, precisely: transparency comes from the IPA's public-coin parameters; accumulation is what makes transparency affordable.
- Two curves, Pallas and Vesta, each the other's scalar field: no secure curve can verify itself natively, so recursion ping-pongs.

“When a Zcash full node verifies a shielded spend, it is running exactly this check — one random evaluation of a committed polynomial identity — and concluding, with cryptographic certainty and learning nothing, that somewhere a valid note was spent by its rightful owner.”

Honest limits

What is still printed on the envelope

For a *fully shielded* transaction, an on-chain observer still sees:

- **Action count** — padded to a floor of 2 with shuffled dummies; above the floor it reveals $\max(\text{spends}, \text{outputs})$ *exactly* (only 1-vs-2 hides);
- **fee** — public by construction (the transparent value left unclaimed); the ZIP-317 formula that keeps it from leaking or fingerprinting is a wallet convention (SHOULD), not consensus;
- **expiry height** — fixed at $\text{target} + 40$, so creation time is coarsely public;
- **anchor and timing** — which tree, when mined, when it hit the mempool.

A sealed envelope with the postmark, weight class, postage, and sell-by date printed on the outside — uniform *only if* every wallet uses the same defaults. The deployed anti-fingerprinting strategy is *conformity*.

The boundary is metered

- Amounts are hidden *inside* a pool; every crossing is a public signed field: the net flow v_{balance} (positive = value leaving the pool) is exact, per transaction.
- Pool migrations (Sprout retirement then, Orchard $\rightarrow$ Ironwood at NU6.3 — testnet-active, mainnet pending) pass through the same public turnstile — per-transaction amounts in the clear, no batching guidance specified.
- Paying a transparent-only recipient from shielded funds (ZIP-320) transits the paid amount through an ephemeral transparent output — visible once, in full.
- Wallet hygiene exists because the boundary leaks: a zero-valued change output is kept even at exact balance, so the *absence* of change never confirms an exact-sum guess.

The server sees your requests

Light-wallet keys never leave the device — *requests do*. The deployed sync protocol leaks, to the server:

observable	reveals	mitigation status
lowest range ever fetched	account birthday	none
range start, each cycle	prior sync height	proposed, unimplemented
full-tx fetch after scan	the wallet's own txids	decoys normative, unimplemented
submission channel	full tx pre-broadcast, + IP	Tor capability, deployment unknown
transparent-address RPCs	addresses outright	jitter (ephemeral only)

- “The server can’t see your transactions” is false as commonly stated: the set of transactions fetched in full *equals* the wallet’s own.
- Structural, not accidental: a client that must ask for ranges reveals where it stopped; a client that must fetch what it detected reveals what it detected.

Frontier (design-stage, one line each)

- **Tachyon** — remove wallet scanning instead of patching its leaks: detection moves to the payment channel, absence-tracking to an oblivious service. The designers' own flagged risk: timing correlation of proof updates — “fatal to the entire approach” if unsolved.
- **Crosslink** — a finality layer under the PoW chain; about settlement assurance, not privacy.

Neither is deployed; nothing here is consensus-specified. The deployed protocol is what the previous thirty slides described.

Takeaways

1. A shielded payment publishes *cryptographic residue*, not data: commitments in, nullifiers out, and a proof as the only bridge.
2. “What do the ZKPs prove?” has a concrete, finite answer: ten checks — note integrity, gated membership, nullifier integrity, value integrity, authority, address binding, consensus gates.
3. Balance needs almost no proof system — a Pedersen sum and one Schnorr signature whose secret key exists only if the books balance, leaning on the circuit’s range checks to keep the sum honest.
4. Keys grade visibility: spend, view-all, view-incoming, receive — auditability is deliberate key handover.
5. Orchard removed the ceremony: public-coin commitments made the trapdoor not secret but *nonexistent*; accumulation made that affordable.
6. The proofs hide the graph; the *systems around them* — fees, boundaries, sync servers — are where privacy actually leaks.

Backup

Backup: Sinsemilla

$$Q(D) := \text{GroupHash}^{\mathbb{G}}(\text{z.cash:SinsemillaQ}, D)$$

$$P[m] := \text{GroupHash}^{\mathbb{G}}(\text{z.cash:SinsemillaS}, \text{LE}_{32}(m))$$

$$\text{Sinsemilla}_D(M) := \text{Acc}_\ell, \quad \text{Acc}_0 := Q(D), \quad \text{Acc}_{i+1} := (\text{Acc}_i \dot{+} P[m_{i+1}]) \dot{+} \text{Acc}_i$$

$$\text{SinsemillaCommit}_r(D, M) := \text{Sinsemilla}_{D \parallel -M}(M) \dot{+} [r] H_D$$

- Each 10-bit chunk costs one table lookup + two incomplete additions; a 510-bit message costs 51 rounds — the chunk-table structure maps directly onto Halo 2's lookup argument.
- Collision resistance (for fixed-length inputs) reduces to discrete log on Pallas, with generator derivation modelled as a random oracle; hiding comes from the uniform $[r]H_D$ term, unconditionally.

Backup: circuit and proof numbers

- One fixed PLONKish circuit, native field $\mathbb{F}_{p_{\text{Pallas}}}$; 2^{11} rows, ten advice columns, Sinsemilla lookup table.
- One aggregate proof per bundle: $2720 + 2272 \cdot n$ bytes for n Actions — 4992 bytes for one Action, including the $2 \log_2 2^{11} = 22$ IPA round elements.
- Action encoding 820 bytes; each signature 64 bytes.
- Prover: FFTs + several tens of $O(n)$ MSMs, seconds on commodity hardware.
Verifier: $O(\log n)$ per proof plus one deferred MSM, amortised over a batch.
- The Merkle gadget stacks 32 layer-tagged hashes; Poseidon's only non-linearity is $x \mapsto x^5$.

Backup: field discipline

lives in	elements	role
$\mathbb{F}_{p_{\text{Vesta}}}$ (Pallas <i>scalar</i> field)	ask, rcm, rcv, r _{ivk} , α , esk, bsk	scalars
$\mathbb{F}_{p_{\text{Pallas}}}$ (Pallas <i>base</i> field)	nk, ρ , ψ , ivk, cmx, nf	field elements
$\mathcal{E}(\mathbb{F}_{p_{\text{Pallas}}})$ (Pallas points)	ak, pk _d , g _d , epk, cm, cv, rk	group elements
raw 256-bit strings	sk, dk, ovk, rseed	keys/seeds

The circuit's native field is $\mathbb{F}_{p_{\text{Pallas}}}$; the containment $p_{\text{Pallas}} < p_{\text{Vesta}}$ is what lets base-field elements act as scalars where needed.

Backup: the note plaintext and rseed

$$\text{np} = (0\text{x}03, d, v, \text{rseed}, \text{memo})$$

lead byte + 11-byte diversifier + 64-bit value + 32-byte seed + 512-byte memo = the 564-byte plaintext; compact prefix = first 52 bytes.

$$\text{esk} := \text{ToScalar}(\text{PRFexpand}_{\text{rseed}}(0\text{x}04 \parallel \rho))$$

$$\psi := \text{ToBase}(\text{PRFexpand}_{\text{rseed}}(0\text{x}09 \parallel \rho))$$

$$\rho := \text{nf}_{\text{old}} \text{ (of the note spent in the same Action)}$$

- Everything the recipient needs re-derives from one 32-byte seed — the ciphertext stays small and the note stays consistent by construction.

Backup: the ZIP-244 sighash

$\text{sighash} := \text{BLAKE2b-256}(\text{ZcashTxHash_} \parallel \text{branchid}, d_{\text{hdr}} \parallel d_{\text{trans}} \parallel d_{\text{Sap}} \parallel d_{\text{Orch}})$

- The 4-byte consensus branch ID keys the personalisation: a signature is valid only on its fork.
- The Orchard branch digests every Action's nullifier, commitment, value commitment, rk, and ciphertexts, plus flags, balance, anchor.
- Nothing can be transplanted or altered without breaking both signature families; for a purely shielded transaction the sighash coincides with the txid.

Backup: the IPA opening in full

$$\mathcal{R}_{\text{IPA}} := \left\{ ((P, x, v); (\vec{a}, r)) : P = \langle \vec{a}, \vec{G} \rangle + [r]H, \quad v = \langle \vec{a}, \vec{b}(x) \rangle, \quad \deg p \leq d-1 \right\}$$

$k = \log_2 d$ halving rounds; final check

$$P + [v] U + \sum_{j=1}^k ([u_j^2] L_j + [u_j^{-2}] R_j) \stackrel{?}{=} [a] G^{(0)} + [r^*] H + [a \cdot b^{(0)}] U$$

- The verifier evaluates the structured polynomial $g(X; u_1, \dots, u_k)$ in $O(\log d)$ — but computing $G^{(0)} = \text{Commit}(g; 0)$ is an $O(d)$ MSM. That asymmetry is the seed of accumulation: $G^{(0)}$ is itself a *commitment* to a structured polynomial, so the deferred check is itself an opening claim that folds into a running accumulator, decided once.

Backup: NU6.3 pool sealing

- Two pools, one protocol: the Orchard *protocol* (curves, hashes, circuit, keys) is shared; the Orchard and Ironwood *pools* each have their own tree, anchor, and nullifier set. Addresses work in both.
- From NU6.3 the legacy Orchard pool is sealed: coinbase must carry an empty Orchard component; every Orchard-pool Action must disable cross-address transfers (A10); and per transaction $v_{\text{balance}}^{\text{Orchard}} \geq 0$ — value exits, never enters.
- The cross-address rule is enforced via the verifying key consensus selects by branch (v5 transactions cannot bypass it); the coinbase and turnstile rules are direct consensus rules on every transaction.

Backup: quantum footnote

- A future discrete-log-breaking adversary who *knows an address* (d, pk_d) computes $ivk = \log_{g_d} pk_d$ and replays the trial-decryption loop over the recorded chain: full receiving history for that address, past and future.
- Two limits: ivk is a viewing capability, not spend authority; and the attack is keyed by the address — without it, the chain reveals nothing even to an unbounded adversary (hiding commitments, uniform re-randomisers, simulatable transcripts, nullifiers keyed by the unpublished nk).